REVISION CHEAT SHEET:

SQL Server Stored Procedures

COMPLETE INTERVIEW-READY NOTES - PARAMETERS - TRANSACTIONS - ERRORS - DYNAMIC SQL - PERFORMANCE

TOPIC & SUBTOPIC INDEX

Study order: fundamentals -> procedure lifecycle -> parameters and return channels -> control flow and temporary processing -> error handling and transactions -> dynamic SQL and pagination -> plan behavior and security -> design best practices -> interview revision.

Why Use Stored Procedures?	Stored Procedure Flow	Stored Procedure vs Normal SQL Query
CREATE PROCEDURE	Why SET NOCOUNT ON?	Executing a Stored Procedure
Procedure Naming and Schemas	Avoid the sp_ Prefix	ALTER PROCEDURE
CREATE OR ALTER PROCEDURE	DROP PROCEDURE	Stored Procedure Parameters
Input Parameters	Multiple Parameters	Passing Parameters by Name
Parameter Data Types	Default Parameters	Optional Parameter Pattern
SQL Server 2025 Optional Parameter Optimization	Parameter Validation	OUTPUT Parameters
Multiple OUTPUT Parameters	RETURN Value	RETURN vs OUTPUT Parameter
Ways a Stored Procedure Can Return Data	Returning a Single Result Set	Returning Multiple Result Sets
Variables Inside Stored Procedures	SET vs SELECT for Variable Assignment	IF / ELSE
CASE	WHILE Loop	Temporary Tables Inside Procedures
#Temp Table vs Table Variable	Error Handling with TRY...CATCH	Error Information Functions
Creating a Custom Error with THROW	Transactions Inside Stored Procedures	Real Fundraising Transaction Example
Transaction Flow	Why SET XACT_ABORT ON?	XACT_STATE()
@@TRANCOUNT	Nested Procedure Transaction Caution	SAVE TRANSACTION
Calling One Procedure from Another	Dynamic SQL	SQL Injection Risk
sp_executesql	EXEC() vs sp_executesql	Dynamic Filters
Dynamic ORDER BY	Pagination with OFFSET / FETCH	Complete Pagination Procedure
Execution Plans and Stored Procedures	Parameter Sniffing	Modern Parameter Sensitive Plan Optimization
WITH RECOMPILE — Procedure Level	OPTION (RECOMPILE)	Procedure Dependencies
Procedure Metadata	Stored Procedure Permissions	Ownership Chaining
EXECUTE AS	System Stored Procedures	Extended Stored Procedures — Historical Legacy Concept
Temporary Stored Procedures	Procedure Design Example — Complete Gift Creation	Stored Procedure Performance Best Practices
Use SARGable Filters	Avoid SELECT *	Keep Transactions Short
Avoid Excessive Business Logic in SQL	Stored Procedure vs View	Stored Procedure vs Function
Stored Procedure vs CTE	Stored Procedure vs Dynamic SQL	Common Stored Procedure Mistakes
Stored Procedure Design Best Practices	Practical Procedure Design Pattern	25 Interview Questions and Practical Problems
Final Revision Map	Final Easy Memory Rules	

A **stored procedure** is a named database program containing one or more Transact-SQL statements that is stored inside SQL Server and executed when needed.

A stored procedure can:

EXAMPLE / FLOW

```
Accept input parameters
Return result sets
Return output parameters
Return an integer status code
Insert / update / delete data
Call other procedures
Use IF / ELSE and loops
Use transactions
Handle errors
Use temporary tables
Execute dynamic SQL
Implement pagination
Apply security permissions
```

Microsoft describes a stored procedure as a group of T-SQL statements, or in some cases a CLR method, that can accept parameters, perform database operations, call other procedures, return output values, and return a status code.

Throughout this chapter, examples use the same `FundraisingDb` model:

EXAMPLE / FLOW

```
core.Tenant
core.AppUser

constituent.Constituent
constituent.Address
constituent.ConstituentAddress

fundraising.Campaign
fundraising.Fund
fundraising.Gift
fundraising.GiftAllocation

engagement.Interaction
engagement.Task
```

The main fundraising relationship remains:

EXAMPLE / FLOW

```
Constituent
     |
     | makes
     v
Gift
     |
     +----------> Campaign
     |
     v
GiftAllocation
     |
     v
Fund
```

Why Use Stored Procedures?

Stored procedures are useful when database operations should be centralized and reusable.

Common reasons include:

EXAMPLE / FLOW

```
Reuse SQL logic

Accept parameters

Encapsulate business-oriented database operations

Perform multiple SQL statements together

Use transactions

Handle errors

Reduce repeated SQL in applications

Control permissions

Return one or more result sets

Improve plan reuse for parameterized operations
```

Example application requirement:

> The .NET Web API needs to retrieve all Gifts for one Constituent.

Instead of embedding this SQL repeatedly:

SQL

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = @ConstituentId;
```

we could create:

EXAMPLE / FLOW

```
fundraising.usp_GetConstituentGifts
```

and call it from the application.

Stored Procedure Flow

EXAMPLE / FLOW

```
.NET / Application
       |
       | sends parameters
       v
Stored Procedure
       |
       | executes T-SQL
       v
SQL Server Tables
       |
       v
Result / Output / Status
       |
       v
Application
```

Stored Procedure vs Normal SQL Query

Normal query:

```sql
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Stored procedure:

```sql
EXEC fundraising.usp_GetConstituentGifts
    @ConstituentId = 1001;
```

The stored procedure provides a named reusable database operation.

CREATE PROCEDURE

Simple Definition

`CREATE PROCEDURE` creates a stored procedure in the database.

Basic syntax:

```sql
CREATE PROCEDURE schema_name.procedure_name
AS
BEGIN
    -- SQL statements
END;
GO
```

Example:

```sql
CREATE PROCEDURE fundraising.usp_GetAllGifts
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        ConstituentId,
        CampaignId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift;
END;
GO
```

Why SET NOCOUNT ON?

You will commonly see:

```sql
SET NOCOUNT ON;
```

near the beginning of stored procedures.

It prevents SQL Server from sending messages such as:

EXAMPLE / FLOW

```
(10 rows affected)
```

after every DML statement.

For application-facing procedures this usually reduces unnecessary informational messages and keeps procedure output cleaner.

Typical pattern:

SQL

```
CREATE PROCEDURE ...
AS
BEGIN
    SET NOCOUNT ON;

    ...
END;
```

Executing a Stored Procedure

Use:

SQL

```
EXEC
```

or:

SQL

```
EXECUTE
```

Both are valid.

Example:

SQL

```
EXEC fundraising.usp_GetAllGifts;
```

or:

SQL

```
EXECUTE fundraising.usp_GetAllGifts;
```

EXEC is simply the shorter form.

Procedure Naming and Schemas

Use meaningful schema-qualified names.

Good examples:

EXAMPLE / FLOW

```
fundraising.usp_GetConstituentGifts

fundraising.usp_CreateGift

engagement.usp_GetOpenTasks

constituent.usp_GetConstituentDetails
```

Always prefer:

```sql
EXEC fundraising.usp_GetConstituentGifts;
```

rather than:

```sql
EXEC usp_GetConstituentGifts;
```

Microsoft recommends schema-qualifying user-defined procedures because it avoids ambiguity and saves SQL Server from searching multiple schemas.

Avoid the sp_ Prefix

Do not name your custom procedures like:

```
sp_GetConstituentGifts
```

Microsoft uses:

```
sp_
```

for system stored procedures.

Use something such as:

```
usp_
```

or simply a meaningful procedure name.

Example:

```
usp_GetConstituentGifts
```

Microsoft specifically recommends avoiding `sp_` for user-defined procedures because it is associated with system procedures and can cause naming conflicts.

ALTER PROCEDURE

Simple Definition

`ALTER PROCEDURE` changes an existing stored procedure without dropping it first.

Example original procedure:

```sql
CREATE PROCEDURE fundraising.usp_GetConstituentGifts
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

Suppose we want to include `CampaignId`.

```sql
ALTER PROCEDURE fundraising.usp_GetConstituentGifts
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        CampaignId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

CREATE OR ALTER PROCEDURE

Modern SQL Server also supports:

```sql
CREATE OR ALTER PROCEDURE
```

This is very convenient for deployment scripts.

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetConstituentGifts
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

If the procedure does not exist:

```
Create it
```

If it already exists:

```
Alter it
```

DROP PROCEDURE

Simple Definition

`DROP PROCEDURE` removes a stored procedure.

```sql
DROP PROCEDURE fundraising.usp_GetConstituentGifts;
GO
```

This removes the procedure definition.

It does **not** delete:

EXAMPLE / FLOW

```
Gift rows

Constituents

Campaigns

Funds
```

Stored Procedure Parameters

Parameters allow the caller to send values into a procedure or receive values back from it.

Main kinds:

EXAMPLE / FLOW

```
Input parameters

Output parameters

Parameters with default values
```

A procedure can have multiple parameters.

SQL Server currently supports up to 2,100 parameters in a procedure, though a good design should normally use far fewer.

Input Parameters

Simple Definition

An **input parameter** sends a value from the caller into the stored procedure.

Example:

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetConstituentGifts
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

Here:

EXAMPLE / FLOW

```
@ConstituentId
```

is an input parameter.

Execute:

SQL

```
EXEC fundraising.usp_GetConstituentGifts
    @ConstituentId = 1001;
```

Flow:

EXAMPLE / FLOW

```
Caller
   |
   | @ConstituentId = 1001
   v
Procedure
   |
   v
Gift table
   |
   v
Gifts for Constituent 1001
```

Multiple Parameters

Suppose we want Gifts for a Constituent during a date range.

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetConstituentGiftsByDate
    @ConstituentId BIGINT,
    @StartDate DATE,
    @EndDate DATE
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId
      AND GiftDate >= @StartDate
      AND GiftDate < DATEADD(DAY, 1, @EndDate);
END;
GO
```

Execute:

SQL

```
EXEC fundraising.usp_GetConstituentGiftsByDate
    @ConstituentId = 1001,
    @StartDate = '2026-01-01',
    @EndDate = '2026-12-31';
```

Passing Parameters by Position

You can write:

SQL

```
EXEC fundraising.usp_GetConstituentGiftsByDate
    1001,
    '2026-01-01',
    '2026-12-31';
```

SQL Server assigns them according to parameter order.

But this is less readable.

Passing Parameters by Name

Prefer:

```sql
EXEC fundraising.usp_GetConstituentGiftsByDate
    @ConstituentId = 1001,
    @StartDate = '2026-01-01',
    @EndDate = '2026-12-31';
```

Benefits:

```
Clearer code

Less confusion about parameter order

Easier maintenance

More readable execution scripts
```

Parameter Data Types

Choose parameter data types that match the related table columns.

Example:

If:

```
Gift.ConstituentId = BIGINT
```

then prefer:

```sql
@ConstituentId BIGINT
```

not:

```sql
@ConstituentId VARCHAR(50)
```

Matching data types helps avoid unnecessary implicit conversion and makes the contract clear.

String Parameter Length

Do not blindly declare:

```sql
@SearchText VARCHAR(MAX)
```

when the related column is much smaller.

Use a reasonable type and length.

Example:

```sql
@CampaignName VARCHAR(200)
```

if that matches the actual business column.

Default Parameters

A procedure parameter can have a default value.

Example:

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetGifts
    @MinimumAmount DECIMAL(18,2) = 0
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE TotalAmount >= @MinimumAmount;
END;
GO
```

Call without parameter:

```sql
EXEC fundraising.usp_GetGifts;
```

Then:

EXAMPLE / FLOW

```
@MinimumAmount = 0
```

Call with parameter:

```sql
EXEC fundraising.usp_GetGifts
    @MinimumAmount = 1000;
```

Now only Gifts of at least $1,000 are returned.

Optional Parameter Pattern

A very common stored procedure pattern is:

```sql
@CampaignId BIGINT = NULL
```

Example:

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_SearchGifts
    @ConstituentId BIGINT = NULL,
    @CampaignId BIGINT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        ConstituentId,
        CampaignId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE
        (@ConstituentId IS NULL
         OR ConstituentId = @ConstituentId)

      AND
        (@CampaignId IS NULL
```

```
        OR CampaignId = @CampaignId);
END;
GO
```

Meaning:

EXAMPLE / FLOW

```
NULL parameter
→ Don't apply that filter

Non-NULL parameter
→ Apply that filter
```

SQL Server 2025 Optional Parameter Optimization

The optional-filter pattern:

SQL

```
WHERE Column = @Parameter
   OR @Parameter IS NULL
```

can suffer from different optimal plans for the filtered and unfiltered cases.

SQL Server 2025 introduces **Optional Parameter Plan Optimization (OPPO)** for eligible queries under compatibility level 170. It can maintain different query variants depending on whether an optional parameter is NULL or non-NULL. It is enabled by default when using compatibility level 170.

This does not mean every optional search procedure is automatically perfect.

Still inspect actual performance.

Parameter Validation

Do not assume callers always send valid data.

Example:

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetConstituentGifts
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    IF @ConstituentId IS NULL
    BEGIN
        THROW 50001,
              'ConstituentId is required.',
              1;
    END;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

Validation Example — Amount

```sql
IF @TotalAmount <= 0
BEGIN
    THROW 50002,
          'Gift amount must be greater than zero.',
          1;
END;
```

Validation belongs in the appropriate layer.

Database constraints should still protect fundamental data integrity.

For example:

```
Procedure validation
+
CHECK constraints
+
Foreign keys
```

can work together.

OUTPUT Parameters

Simple Definition

An `OUTPUT` parameter sends a value from the stored procedure back to the caller.

Example requirement:

> Return the lifetime giving total for one Constituent through an output parameter.

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetConstituentGivingTotal
    @ConstituentId BIGINT,
    @LifetimeGiving DECIMAL(18,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        @LifetimeGiving =
            COALESCE(SUM(TotalAmount), 0)
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

Execute:

```sql
DECLARE @Total DECIMAL(18,2);

EXEC fundraising.usp_GetConstituentGivingTotal
    @ConstituentId = 1001,
    @LifetimeGiving = @Total OUTPUT;

SELECT @Total AS LifetimeGiving;
```

OUTPUT Must Be Declared Twice

In procedure definition:

```sql
@LifetimeGiving DECIMAL(18,2) OUTPUT
```

When capturing it during execution:

```sql
@LifetimeGiving = @Total OUTPUT
```

If the caller does not use **OUTPUT**, the procedure can still execute, but the caller will not capture the returned output value.

Multiple OUTPUT Parameters

Example:

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetConstituentGivingStats
    @ConstituentId BIGINT,
    @GiftCount INT OUTPUT,
    @LifetimeGiving DECIMAL(18,2) OUTPUT,
    @LargestGift DECIMAL(18,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        @GiftCount = COUNT(*),
        @LifetimeGiving =
            COALESCE(SUM(TotalAmount), 0),
        @LargestGift =
            COALESCE(MAX(TotalAmount), 0)
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

RETURN Value

Every stored procedure can return an integer status code.

Example:

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_CheckConstituent
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS
    (
        SELECT 1
        FROM constituent.Constituent
        WHERE ConstituentId = @ConstituentId
    )
    BEGIN
        RETURN 1;
    END;

    RETURN 0;
END;
GO
```

Capture it:

SQL

```
DECLARE @ReturnCode INT;

EXEC @ReturnCode =
    fundraising.usp_CheckConstituent
        @ConstituentId = 1001;

SELECT @ReturnCode AS ReturnCode;
```

SQL Server procedure return codes are integers and are traditionally used to communicate execution status. If a procedure does not explicitly return a value, its return code is normally `0`.

RETURN vs OUTPUT Parameter

This is an important interview topic.

RETURN	OUTPUT Parameter
Integer only	Many supported data types
Usually status code	Usually business/data value
One return code	Multiple OUTPUT parameters possible
`RETURN 0` etc.	`@Total OUTPUT`

Use:

EXAMPLE / FLOW

```
RETURN
→ success/failure/status
```

Use:

EXAMPLE / FLOW

```
OUTPUT
→ actual returned data values
```

Do not use:

EXAMPLE / FLOW

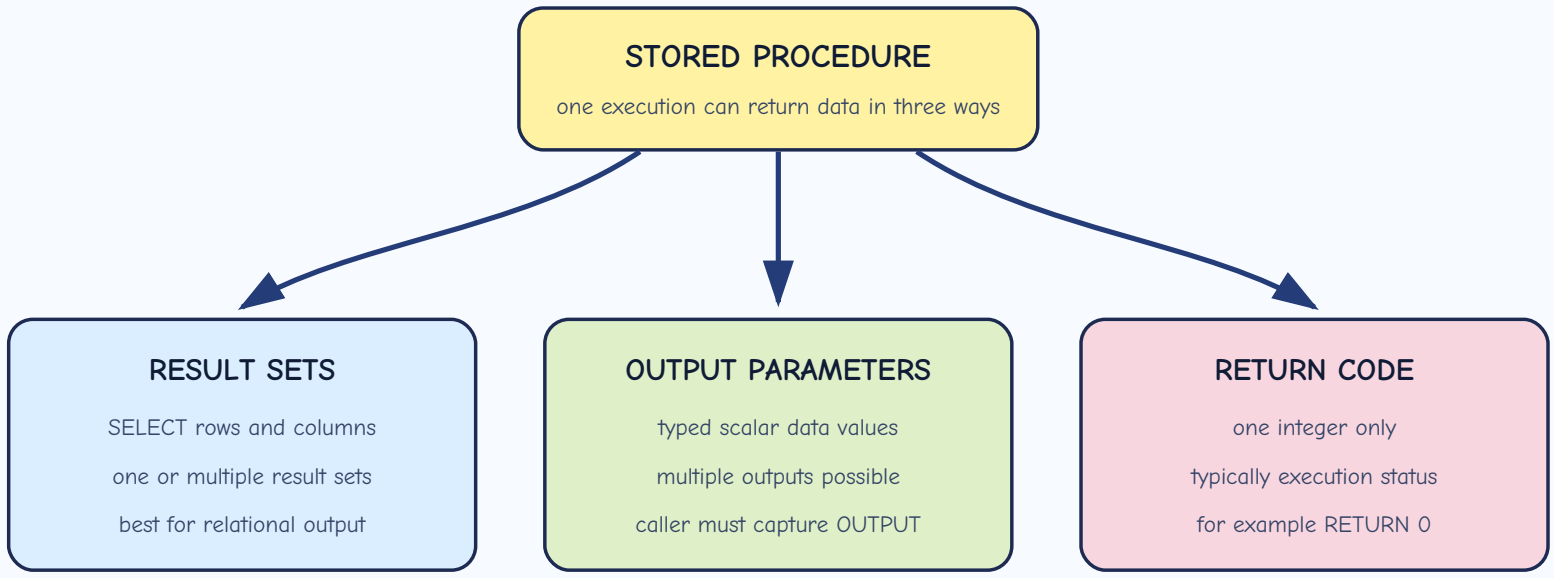
```
RETURN @LifetimeGiving
```

for monetary data.

Ways a Stored Procedure Can Return Data

3D DIAGRAM

How Stored Procedures Return Information



Choose the return mechanism based on meaning: rows, scalar business values, or an integer status code.

SQL Server procedures commonly return data in three ways:

EXAMPLE / FLOW

```
Result sets

Output parameters

Integer return codes
```

Microsoft documents these three mechanisms explicitly.

Returning a Single Result Set

Example:

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetCampaignGifts
    @CampaignId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE CampaignId = @CampaignId
    ORDER BY
        GiftDate DESC,
        GiftId DESC;
END;
GO
```

One **SELECT** returns one result set.

Returning Multiple Result Sets

A procedure can contain several **SELECT** statements.

Example:

SQL

```
CREATE OR ALTER PROCEDURE constituent.usp_GetConstituentProfile
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    -- Result set 1
    SELECT
        ConstituentId,
        DisplayName,
        PrimaryEmail,
        PrimaryPhone
    FROM constituent.Constituent
    WHERE ConstituentId = @ConstituentId;

    -- Result set 2
    SELECT
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId
    ORDER BY
        GiftDate DESC,
        GiftId DESC;

    -- Result set 3
    SELECT
        InteractionId,
        InteractionDate,
        InteractionType,
        Subject
    FROM engagement.Interaction
    WHERE ConstituentId = @ConstituentId
    ORDER BY
        InteractionDate DESC,
        InteractionId DESC;
END;
GO
```

The caller receives:

EXAMPLE / FLOW

```
Result Set 1
→ Constituent details

Result Set 2
→ Gifts

Result Set 3
→ Interactions
```

When to Return Multiple Result Sets

Useful when:

EXAMPLE / FLOW

```
One database call naturally returns related sections

Application data-access layer supports multiple result sets
```

```
The pieces belong to one operation
```

Avoid using many unrelated result sets in a single procedure just to reduce calls.

That can make the contract difficult to maintain.

Variables Inside Stored Procedures

Variables store temporary scalar values during procedure execution.

Syntax:

```sql
DECLARE @VariableName DataType;
```

Example:

```sql
DECLARE @LifetimeGiving DECIMAL(18,2);
```

Assign:

```sql
SELECT
    @LifetimeGiving =
        SUM(TotalAmount)
FROM fundraising.Gift
WHERE ConstituentId = @ConstituentId;
```

SET vs SELECT for Variable Assignment

You can use:

```sql
SET @Variable = value;
```

or:

```sql
SELECT @Variable = expression;
```

Example:

```sql
SET @MinimumAmount = 1000;
```

or:

```sql
SELECT
    @GiftCount = COUNT(*)
FROM fundraising.Gift;
```

For a simple scalar assignment, **SET** is very clear.

For assigning several variables from one query, **SELECT** is convenient.

IF / ELSE

Simple Definition

`IF / ELSE` performs different logic depending on a condition.

Example:

```sql
IF @MinimumAmount < 0
BEGIN
    THROW 50003,
          'MinimumAmount cannot be negative.',
          1;
END;
ELSE
BEGIN
    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift
    WHERE TotalAmount >= @MinimumAmount;
END;
```

Real Fundraising Example

Suppose:

```
@IncludeInactive = 1
```

means include every Campaign.

Otherwise only active Campaigns.

```sql
IF @IncludeInactive = 1
BEGIN
    SELECT
        CampaignId,
        CampaignName,
        Status
    FROM fundraising.Campaign;
END;
ELSE
BEGIN
    SELECT
        CampaignId,
        CampaignName,
        Status
    FROM fundraising.Campaign
    WHERE Status = 'Active';
END;
```

CASE

`CASE` is different from `IF`.

`IF` controls which statements execute.

`CASE` returns a value inside an expression.

Example:

```sql
SELECT
    GiftId,
    TotalAmount,
```

```
    CASE
        WHEN TotalAmount >= 10000
            THEN 'Major Gift'

        WHEN TotalAmount >= 1000
            THEN 'Large Gift'

        ELSE 'Standard Gift'
    END AS GiftCategory

FROM fundraising.Gift;
```

Memory:

EXAMPLE / FLOW

```
IF
→ Control flow

CASE
→ Conditional expression
```

WHILE Loop

Simple Definition

WHILE repeats statements while a condition remains true.

Example:

SQL

```
DECLARE @Counter INT = 1;

WHILE @Counter <= 5
BEGIN
    PRINT @Counter;

    SET @Counter = @Counter + 1;
END;
```

Should WHILE Be Used for Normal Data Processing?

Usually prefer set-based SQL.

Less ideal:

EXAMPLE / FLOW

```
Loop through 1,000 Gifts
one Gift at a time
```

Better when possible:

SQL

```
UPDATE fundraising.Gift
SET ...
WHERE ...;
```

SQL Server is designed to process sets efficiently.

Use loops when the operation genuinely requires sequential logic.

Temporary Tables Inside Procedures

A procedure can create a local temporary table.

Example:

```sql
CREATE TABLE #ConstituentGiving
(
    ConstituentId BIGINT NOT NULL,
    GiftCount INT NOT NULL,
    LifetimeGiving DECIMAL(18,2) NOT NULL
);
```

Populate:

```sql
INSERT INTO #ConstituentGiving
(
    ConstituentId,
    GiftCount,
    LifetimeGiving
)
SELECT
    ConstituentId,
    COUNT(*),
    SUM(TotalAmount)
FROM fundraising.Gift
GROUP BY ConstituentId;
```

Then query it:

```sql
SELECT *
FROM #ConstituentGiving;
```

Why Use #Temp Tables in Procedures?

Useful when:

```
Intermediate result is reused

Processing has several stages

You need temporary indexes

Intermediate data is large

Breaking a complex query improves plan quality

You need statistics on intermediate rows
```

Temp Table Scope in Procedures

A local temp table created inside a stored procedure is automatically dropped when that procedure finishes.

A nested stored procedure called from it can access that temp table.

However, the procedure's caller cannot use a temp table that was created only inside the procedure after the procedure returns.

Flow:

EXAMPLE / FLOW

```
Procedure A
   |
   | creates #GiftSummary
   |
   +---- calls Procedure B
   |       |
   |       +---- can see #GiftSummary
   |
   v
Procedure A finishes
   |
   v
#GiftSummary automatically dropped
```

Caller-Created Temp Table

If the outer session creates:

SQL

```
CREATE TABLE #GiftIds
(
    GiftId BIGINT
);
```

and then executes a stored procedure, the procedure can normally see that session-level local temp table.

This can be useful but creates hidden coupling between caller and procedure.

Use carefully.

Table Variables Inside Procedures

A table variable is declared with:

SQL

```
DECLARE @GiftSummary TABLE
(
    ConstituentId BIGINT,
    GiftCount INT,
    LifetimeGiving DECIMAL(18,2)
);
```

Insert:

SQL

```
INSERT INTO @GiftSummary
(
    ConstituentId,
    GiftCount,
    LifetimeGiving
)
SELECT
    ConstituentId,
    COUNT(*),
    SUM(TotalAmount)
FROM fundraising.Gift
GROUP BY ConstituentId;
```

#Temp Table vs Table Variable

#Temp Table	Table Variable
Temporary table in tempdb	Table variable
Can have indexes	Can define keys/indexes through declarations
Has richer statistics behavior	Different optimization behavior
Good for larger/reused intermediate sets	Often convenient for smaller/simple sets
Scope may include nested procedures	Variable scope stays in current batch/module
Can be explicitly dropped	Automatically disappears with scope

Do not use the simplistic rule:

EXAMPLE / FLOW

```
"Table variables are in memory,
temp tables are on disk."
```

Both can involve **tempdb**.

Choose based on workload and execution-plan behavior.

Error Handling with TRY...CATCH

A stored procedure should handle expected database errors in a controlled way.

Basic pattern:

SQL

```
BEGIN TRY

    -- statements

END TRY
BEGIN CATCH

    -- error handling

END CATCH;
```

Error Information Functions

Inside **CATCH**, SQL Server provides:

EXAMPLE / FLOW

```
ERROR_NUMBER()

ERROR_MESSAGE()

ERROR_SEVERITY()

ERROR_STATE()

ERROR_LINE()

ERROR_PROCEDURE()
```

Microsoft documents these functions specifically for retrieving information about the error caught by **TRY...CATCH**.

Error Handling Example

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_CreateGift
    @TenantId BIGINT,
    @ConstituentId BIGINT,
    @CampaignId BIGINT = NULL,
    @GiftDate DATE,
    @TotalAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY

        IF @TotalAmount <= 0
        BEGIN
            THROW 50010,
                  'TotalAmount must be greater than zero.',
                  1;
        END;

        INSERT INTO fundraising.Gift
        (
            TenantId,
            ConstituentId,
            CampaignId,
            GiftDate,
            TotalAmount
        )
        VALUES
        (
            @TenantId,
            @ConstituentId,
            @CampaignId,
            @GiftDate,
            @TotalAmount
        );

    END TRY
    BEGIN CATCH

        SELECT
            ERROR_NUMBER() AS ErrorNumber,
            ERROR_MESSAGE() AS ErrorMessage,
            ERROR_SEVERITY() AS ErrorSeverity,
            ERROR_STATE() AS ErrorState,
            ERROR_LINE() AS ErrorLine,
            ERROR_PROCEDURE() AS ErrorProcedure;

        THROW;

    END CATCH;
END;
GO
```

Why THROW Again?

Inside **CATCH**:

SQL

```
THROW;
```

rethrows the original error.

This is usually better than silently swallowing the error.

The caller can then:

EXAMPLE / FLOW

```
Log the failure

Return appropriate API response

Roll back higher-level work if needed
```

A parameterless **THROW** is valid inside **CATCH** and rethrows the caught exception.

Creating a Custom Error with THROW

Syntax:

SQL

```
THROW error_number,
      message,
      state;
```

Example:

SQL

```
THROW 50020,
      'CampaignId does not exist.',
      1;
```

Custom **THROW** error numbers must be at least 50000.

Error Handling Best Practices

A good procedure generally should:

EXAMPLE / FLOW

```
Validate important parameters

Use TRY...CATCH around transactional work

Roll back failed transactions

Use THROW so callers know the operation failed

Do not hide errors unnecessarily

Log important operational failures at the appropriate layer

Return status data consistently
```

Transactions Inside Stored Procedures

A transaction groups operations so they succeed or fail together.

Example fundraising requirement:



Create a Gift and its GiftAllocation rows as one unit.

You do not want:

EXAMPLE / FLOW

```
Gift inserted successfully
```

```
but

GiftAllocation insertion fails
```

leaving incomplete financial data.

Basic Transaction Structure

```sql
BEGIN TRANSACTION;

-- work

COMMIT TRANSACTION;
```

If something goes wrong:

```sql
ROLLBACK TRANSACTION;
```

Real Fundraising Transaction Example

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_CreateGiftWithAllocation
    @TenantId BIGINT,
    @ConstituentId BIGINT,
    @CampaignId BIGINT = NULL,
    @FundId BIGINT,
    @GiftDate DATE,
    @TotalAmount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY

        BEGIN TRANSACTION;

        INSERT INTO fundraising.Gift
        (
            TenantId,
            ConstituentId,
            CampaignId,
            GiftDate,
            TotalAmount
        )
        VALUES
        (
            @TenantId,
            @ConstituentId,
            @CampaignId,
            @GiftDate,
            @TotalAmount
        );

        DECLARE @GiftId BIGINT =
            SCOPE_IDENTITY();

        INSERT INTO fundraising.GiftAllocation
        (
            TenantId,
```

```
            GiftId,
            FundId,
            AllocationAmount
        )
        VALUES
        (
            @TenantId,
            @GiftId,
            @FundId,
            @TotalAmount
        );

        COMMIT TRANSACTION;

        SELECT
            @GiftId AS GiftId;

    END TRY
    BEGIN CATCH

        IF XACT_STATE() <> 0
        BEGIN
            ROLLBACK TRANSACTION;
        END;

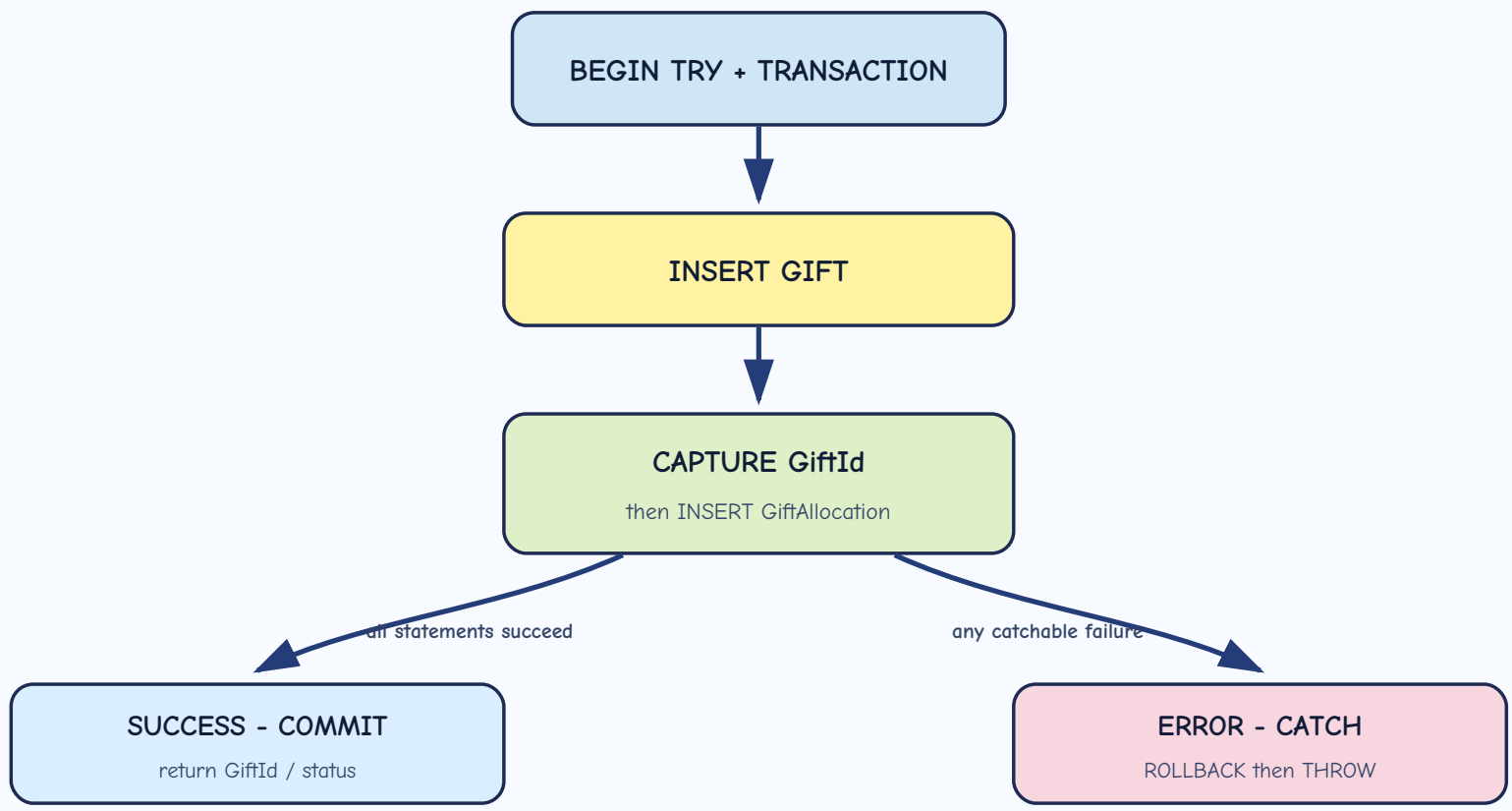
        THROW;

    END CATCH;
END;
GO
```

Transaction Flow

3D DIAGRAM

Safe Gift Creation - Transaction and Error Path



Gift and allocation belong to one atomic business operation: commit together or roll back together.

EXAMPLE / FLOW

```
BEGIN TRANSACTION
       |
       v
Insert Gift
       |
       v
Insert GiftAllocation
       |
       +---- success ----> COMMIT
       |
       +---- failure ----> ROLLBACK
```

Why SET XACT_ABORT ON?

A common robust transaction pattern is:

SQL

```
SET XACT_ABORT ON;
```

combined with:

EXAMPLE / FLOW

```
TRY...CATCH
+
XACT_STATE()
```

Certain run-time errors can make a transaction uncommittable. Microsoft documents `XACT_STATE()` as the way to distinguish whether there is no transaction, a committable transaction, or an uncommittable transaction.

XACT_STATE()

`XACT_STATE()` returns:

EXAMPLE / FLOW

```
1
→ Active and committable transaction

0
→ No active transaction

-1
→ Active but uncommittable transaction
```

Typical CATCH logic:

SQL

```
IF XACT_STATE() <> 0
BEGIN
    ROLLBACK TRANSACTION;
END;
```

@@TRANCOUNT

`@@TRANCOUNT` tells you how many transaction levels are currently active.

Example:

```sql
SELECT @@TRANCOUNT;
```

Useful when procedures might be called inside an already active transaction.

Nested Procedure Transaction Caution

Suppose:

```
Procedure A
    |
    +---- BEGIN TRANSACTION
    |
    +---- calls Procedure B
```

If Procedure B blindly:

```sql
ROLLBACK TRANSACTION;
```

it can roll back the entire transaction, not merely its own logical work.

Stored procedure transaction design must consider:

```
Was a transaction already active?

Who owns the transaction?

Should we use a savepoint?

Who should commit?
```

SAVE TRANSACTION

A savepoint marks a location inside a transaction.

Example:

```sql
SAVE TRANSACTION BeforeAllocations;
```

Then:

```sql
ROLLBACK TRANSACTION BeforeAllocations;
```

can roll back to that savepoint when the transaction remains committable.

This does not end the outer transaction.

Savepoint Example

```sql
BEGIN TRANSACTION;

INSERT INTO fundraising.Gift (...);

SAVE TRANSACTION GiftCreated;
```

```
BEGIN TRY

    INSERT INTO fundraising.GiftAllocation (...);

END TRY
BEGIN CATCH

    IF XACT_STATE() = 1
    BEGIN
        ROLLBACK TRANSACTION GiftCreated;
    END;

END CATCH;
```

Use savepoints only when partial rollback is genuinely part of the business design.

For most atomic business operations:

EXAMPLE / FLOW

```
all succeed
or
all roll back
```

is easier and safer.

Calling One Procedure from Another

A procedure can execute another procedure.

Example:

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_ProcessConstituent
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    EXEC fundraising.usp_GetConstituentGifts
        @ConstituentId = @ConstituentId;
END;
GO
```

This can be useful for modular logic.

Avoid Excessive Procedure Nesting

Example:

EXAMPLE / FLOW

```
Procedure A
  ↓
Procedure B
  ↓
Procedure C
  ↓
Procedure D
```

can become hard to:

EXAMPLE / FLOW

```
Debug

Understand transaction ownership

Trace errors

Understand temp-table dependencies

Tune performance
```

Modularity is useful, but avoid unnecessary layers.

Dynamic SQL

Simple Definition

Dynamic SQL means constructing SQL text at runtime and executing it.

Example concept:

EXAMPLE / FLOW

```
Normal SQL
→ Query known at design time

Dynamic SQL
→ Query text changes at runtime
```

EXEC() Dynamic SQL

Example:

SQL

```
DECLARE @Sql NVARCHAR(MAX);

SET @Sql =
N'SELECT
      GiftId,
      GiftDate,
      TotalAmount
  FROM fundraising.Gift;';

EXEC(@Sql);
```

This executes the SQL stored in the string.

Why Use Dynamic SQL?

Valid scenarios include:

EXAMPLE / FLOW

```
Dynamic table/column names

Dynamic ORDER BY

Optional complex search predicates

Administrative scripts

Metadata-driven processing
```

Do not use dynamic SQL just because ordinary SQL seems inconvenient.

SQL Injection Risk

This is dangerous:

```sql
SET @Sql =
N'SELECT *
  FROM fundraising.Campaign
  WHERE CampaignName = '''
  + @CampaignName
  + N''';';
```

If untrusted input is concatenated into executable SQL, an attacker may alter the command.

Microsoft recommends parameterizing dynamic SQL and not concatenating user input directly into executable SQL text.

sp_executesql

For parameterized dynamic SQL, prefer:

EXAMPLE / FLOW

```
sys.sp_executesql
```

Example:

```sql
DECLARE @Sql NVARCHAR(MAX);

SET @Sql =
N'
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = @Id;
';

EXEC sys.sp_executesql
    @Sql,
    N'@Id BIGINT',
    @Id = @ConstituentId;
```

The parameter value remains separate from the SQL text.

`sp_executesql` can also improve plan reuse when the SQL text stays constant while only parameter values change.

EXEC() vs sp_executesql

EXEC(@sql)	sp_executesql
Executes dynamic string	Executes dynamic parameterized batch
Easy to misuse with concatenation	Supports typed parameters
Lower plan reuse potential in many patterns	Better opportunity for plan reuse
Greater injection risk when badly built	Safer when values are parameterized

Prefer:

EXAMPLE / FLOW

```
sp_executesql
```

for dynamic SQL involving data values.

Dynamic Filters

Suppose users can filter Gifts by:

EXAMPLE / FLOW

```
Constituent

Campaign

Minimum amount
```

One static approach:

SQL

```
WHERE
    (@ConstituentId IS NULL
     OR ConstituentId = @ConstituentId)

AND
    (@CampaignId IS NULL
     OR CampaignId = @CampaignId)

AND
    (@MinimumAmount IS NULL
     OR TotalAmount >= @MinimumAmount)
```

This is simple and maintainable.

For more complicated combinations, dynamic SQL may build only the predicates that are needed.

Parameterized Dynamic Search Example

SQL

```
DECLARE @Sql NVARCHAR(MAX) =
N'
SELECT
    GiftId,
    ConstituentId,
    CampaignId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE 1 = 1
';

IF @ConstituentId IS NOT NULL
BEGIN
    SET @Sql +=
        N' AND ConstituentId = @pConstituentId';
END;

IF @CampaignId IS NOT NULL
BEGIN
    SET @Sql +=
        N' AND CampaignId = @pCampaignId';
END;

EXEC sys.sp_executesql
    @Sql,
    N'
      @pConstituentId BIGINT,
      @pCampaignId BIGINT
```

```
    ',
    @pConstituentId = @ConstituentId,
    @pCampaignId = @CampaignId;
```

Notice:

EXAMPLE / FLOW

```
SQL structure
→ constructed

Parameter values
→ still parameterized
```

Dynamic ORDER BY

You cannot use a parameter directly in place of a column name like this:

SQL

```
ORDER BY @SortColumn;
```

and expect SQL Server to interpret `@SortColumn = 'GiftDate'` as a column identifier.

Parameters represent:

EXAMPLE / FLOW

```
values
```

not:

EXAMPLE / FLOW

```
table names
column names
keywords
```

Microsoft explicitly notes that procedure parameters cannot replace object names such as table or column names.

Safe Static ORDER BY with CASE

For a small known list of sort choices:

SQL

```
ORDER BY

    CASE
        WHEN @SortColumn = 'GiftDate'
             AND @SortDirection = 'ASC'
        THEN GiftDate
    END ASC,

    CASE
        WHEN @SortColumn = 'GiftDate'
             AND @SortDirection = 'DESC'
        THEN GiftDate
    END DESC;
```

This avoids dynamic SQL but becomes verbose when there are many options.

Dynamic ORDER BY with Whitelist

If dynamic SQL is necessary:

```sql
IF @SortColumn NOT IN
(
    'GiftDate',
    'TotalAmount',
    'GiftId'
)
BEGIN
    THROW 50030,
          'Invalid sort column.',
          1;
END;
```

Then safely construct only from known values.

Do not concatenate arbitrary user-entered column names.

For identifier quoting, `QUOTENAME()` is useful after whitelist validation.

Pagination with OFFSET / FETCH

Stored procedures often implement pagination for APIs.

Example parameters:

```sql
@PageNumber INT = 1,
@PageSize INT = 25
```

Calculate offset:

```sql
DECLARE @Offset INT =
    (@PageNumber - 1)
    * @PageSize;
```

Query:

```sql
SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM fundraising.Gift

ORDER BY
    GiftDate DESC,
    GiftId DESC

OFFSET @Offset ROWS

FETCH NEXT @PageSize ROWS ONLY;
```

Complete Pagination Procedure

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_GetGiftsPaged
    @PageNumber INT = 1,
    @PageSize INT = 25
AS
BEGIN
```

```
    SET NOCOUNT ON;

    IF @PageNumber < 1
    BEGIN
        THROW 50040,
              'PageNumber must be at least 1.',
              1;
    END;

    IF @PageSize < 1
       OR @PageSize > 200
    BEGIN
        THROW 50041,
              'PageSize must be between 1 and 200.',
              1;
    END;

    DECLARE @Offset INT =
        (@PageNumber - 1)
        * @PageSize;

    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    ORDER BY
        GiftDate DESC,
        GiftId DESC

    OFFSET @Offset ROWS
    FETCH NEXT @PageSize ROWS ONLY;
END;
GO
```

Why Use a Unique Tie-Breaker in Pagination?

Suppose ten Gifts have the same:

EXAMPLE / FLOW

```
GiftDate
```

Ordering only by:

SQL

```
ORDER BY GiftDate DESC
```

does not fully determine row order.

Add:

EXAMPLE / FLOW

```
GiftId DESC
```

to make ordering stable.

Microsoft notes that stable paging requires deterministic ordering, and changes to underlying data between page requests can still affect results.

OFFSET Pagination Performance

For small page numbers:

EXAMPLE / FLOW

```
OFFSET/FETCH
```

is straightforward.

For very deep pages:

EXAMPLE / FLOW

```
Page 50,000
```

SQL Server may need to process or skip many earlier rows.

For high-volume APIs, consider:

EXAMPLE / FLOW

```
Keyset / seek pagination
```

using the last seen key instead of a very large `OFFSET`.

That is a later performance topic.

Execution Plans and Stored Procedures

When SQL Server executes a stored procedure, statements are compiled into execution plans.

SQL Server can cache and reuse plans for later executions.

This is one reason parameterized stored procedures can perform efficiently.

However:

EXAMPLE / FLOW

```
one cached plan
```

may not be optimal for every possible parameter value.

This leads to:

EXAMPLE / FLOW

```
parameter sensitivity
```

commonly discussed as:

EXAMPLE / FLOW

```
parameter sniffing
```

Parameter Sniffing

Simple Definition

When a parameterized statement is compiled, SQL Server can use the current parameter value when estimating row counts and choosing a plan.

Example procedure:

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetGiftsByConstituent
    @ConstituentId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
```

```
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
```

Suppose:

EXAMPLE / FLOW

```
Constituent A
→ 2 Gifts

Constituent B
→ 2,000,000 Gifts
```

The best plan may not be the same for both.

Is Parameter Sniffing Always Bad?

No.

Parameter sniffing is normally useful because it lets SQL Server optimize using real parameter values.

The problem occurs when:

EXAMPLE / FLOW

```
data distribution is highly skewed
```

and a plan compiled for one type of value performs poorly for another.

Microsoft specifically describes `WITH RECOMPILE` as one possible mitigation when parameter values are atypical.

Modern Parameter Sensitive Plan Optimization

SQL Server 2022 introduced Parameter Sensitive Plan optimization for eligible queries, allowing multiple cached plans for different parameter-value ranges.

SQL Server 2025 extends this adaptive-plan approach and also adds OPPO for optional-parameter predicates.

Therefore:

> Do not automatically add `RECOMPILE` to every parameterized procedure.

First inspect whether there is an actual parameter-sensitive performance problem.

WITH RECOMPILE — Procedure Level

Example:

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetGiftsByConstituent
    @ConstituentId BIGINT

WITH RECOMPILE

AS
BEGIN
    SET NOCOUNT ON;

    SELECT
```

```
        GiftId,
        GiftDate,
        TotalAmount
    FROM fundraising.Gift
    WHERE ConstituentId = @ConstituentId;
END;
GO
```

This causes SQL Server to compile a new plan for procedure execution rather than reusing a cached plan for the procedure.

Execute Once WITH RECOMPILE

You can also request recompilation for a particular execution:

SQL

```
EXEC fundraising.usp_GetGiftsByConstituent
    @ConstituentId = 1001
WITH RECOMPILE;
```

OPTION (RECOMPILE)

Often only one statement needs recompilation.

Example:

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount

FROM fundraising.Gift

WHERE ConstituentId = @ConstituentId

OPTION (RECOMPILE);
```

This recompiles that statement for the current execution.

Microsoft notes that statement-level `OPTION (RECOMPILE)` can be preferable when only a particular statement has parameter-sensitive behavior.

RECOMPILE Trade-Off

Benefit:

EXAMPLE / FLOW

```
Plan optimized using current values
```

Cost:

EXAMPLE / FLOW

```
Compilation CPU every execution
Less plan reuse
```

Use only when justified by measured performance.

Procedure Dependencies

A stored procedure can depend on:

EXAMPLE / FLOW

```
Tables

Views

Functions

Other procedures
```

Example:

EXAMPLE / FLOW

```
fundraising.usp_GetConstituentGifts
        |
        v
fundraising.Gift
```

Finding Dependencies

Example:

SQL

```
SELECT
    referenced_schema_name,
    referenced_entity_name

FROM sys.sql_expression_dependencies

WHERE referencing_id =
    OBJECT_ID
    (
        'fundraising.usp_GetConstituentGifts'
    );
```

Another option:

SQL

```
SELECT *
FROM sys.dm_sql_referenced_entities
(
    'fundraising.usp_GetConstituentGifts',
    'OBJECT'
);
```

Dependency analysis is useful before:

EXAMPLE / FLOW

```
Dropping columns

Renaming tables

Changing data types

Refactoring database objects
```

Procedure Metadata

Find procedures:

```sql
SELECT
    SCHEMA_NAME(schema_id)
        AS SchemaName,

    name AS ProcedureName,

    create_date,

    modify_date

FROM sys.procedures

ORDER BY
    SchemaName,
    ProcedureName;
```

View Procedure Definition

```sql
SELECT
    OBJECT_DEFINITION
    (
        OBJECT_ID
        (
            'fundraising.usp_GetConstituentGifts'
        )
    );
```

or:

```sql
EXEC sys.sp_helptext
    N'fundraising.usp_GetConstituentGifts';
```

Stored Procedure Permissions

A user can be granted permission to execute a procedure.

Example:

```sql
GRANT EXECUTE
ON OBJECT::fundraising.usp_GetConstituentGifts
TO FundraisingAppRole;
```

This allows access to the procedure without necessarily granting broad direct permissions on every underlying table, depending on ownership chaining and security design.

Why Procedure Security Is Useful

Instead of granting an application:

```
SELECT / INSERT / UPDATE / DELETE
on many tables
```

you may grant:

EXAMPLE / FLOW

```
EXECUTE
on specific procedures
```

Example:

EXAMPLE / FLOW

```
Application can execute:

usp_CreateGift
usp_GetConstituentGifts
usp_UpdateTaskStatus

but cannot freely:
DELETE FROM fundraising.Gift
```

This can support least-privilege design.

Ownership Chaining

If the procedure and referenced objects have compatible ownership, SQL Server may not need to re-check underlying object permissions for normal DML access.

However:

💡 Dynamic SQL does not automatically receive the same ownership-chaining behavior.

Microsoft explicitly documents that ownership chaining does not apply to dynamic queries inside the module.

This is important when combining:

EXAMPLE / FLOW

```
Stored procedure security
+
dynamic SQL
```

EXECUTE AS

A procedure can specify a security execution context.

Example concept:

SQL

```
CREATE PROCEDURE ...
WITH EXECUTE AS OWNER
AS
...
```

Possible contexts include forms such as:

EXAMPLE / FLOW

```
CALLER

SELF

OWNER

specific database principal
```

Use this only as part of deliberate security design.

Do not use `EXECUTE AS OWNER` simply to bypass permissions without understanding the impact.

System Stored Procedures

SQL Server includes many built-in system stored procedures.

Examples:

EXAMPLE / FLOW

```
sys.sp_help

sys.sp_helptext

sys.sp_executesql

sys.sp_refreshview

sys.sp_recompile
```

System procedures are logically exposed through the:

EXAMPLE / FLOW

```
sys
```

schema. Microsoft recommends explicitly using that schema when calling system procedures.

Example:

SQL

```
EXEC sys.sp_help
    N'fundraising.Gift';
```

sp_executesql

This important system procedure executes parameterized dynamic T-SQL.

SQL

```
EXEC sys.sp_executesql
    @Sql,
    N'@Id BIGINT',
    @Id = 1001;
```

It is commonly used inside procedures that need dynamic search or sorting logic.

Extended Stored Procedures — Historical Legacy Concept

Older SQL Server versions support a technology called:

EXAMPLE / FLOW

```
Extended Stored Procedures
```

These were native DLL-based routines written using the Extended Stored Procedure API.

They are **not** normal T-SQL stored procedures.

Historically they allowed code outside the SQL Database Engine to extend SQL Server functionality.

Current Microsoft guidance states that extended stored procedures will be removed in a future version and should not be used for new development; existing solutions should migrate to newer technologies such as CLR integration where appropriate.

For modern application development:

EXAMPLE / FLOW

```
Know what they are for interviews/legacy maintenance.

Do not choose them for new development.
```

Temporary Stored Procedures

SQL Server can create:

EXAMPLE / FLOW

```
#LocalTemporaryProcedure

##GlobalTemporaryProcedure
```

These are specialized and rarely needed in normal application design.

Example:

SQL

```
CREATE PROCEDURE #usp_TempExample
AS
BEGIN
    SELECT 1 AS Example;
END;
```

A local temporary procedure lasts for the creating connection.

Microsoft also documents global temporary procedures, though they are uncommon in modern application development.

Procedure Design Example — Complete Gift Creation

Now combine several stored-procedure concepts.

Requirement:

> Create a Gift allocated entirely to one Fund.

We want:

EXAMPLE / FLOW

```
Input parameters

Validation

Transaction

TRY...CATCH

Gift insert

GiftAllocation insert

Output parameter

Return status
```

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_CreateGift
    @TenantId BIGINT,
```

```
    @ConstituentId BIGINT,
    @CampaignId BIGINT = NULL,
    @FundId BIGINT,
    @GiftDate DATE,
    @TotalAmount DECIMAL(18,2),
    @NewGiftId BIGINT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    SET @NewGiftId = NULL;

    IF @TenantId IS NULL
       OR @ConstituentId IS NULL
       OR @FundId IS NULL
    BEGIN
        THROW 50100,
              'TenantId, ConstituentId and FundId are required.',
              1;
    END;

    IF @TotalAmount <= 0
    BEGIN
        THROW 50101,
              'TotalAmount must be greater than zero.',
              1;
    END;

    IF NOT EXISTS
    (
        SELECT 1
        FROM constituent.Constituent
        WHERE ConstituentId = @ConstituentId
          AND TenantId = @TenantId
    )
    BEGIN
        THROW 50102,
              'Constituent does not exist for the tenant.',
              1;
    END;

    BEGIN TRY

        BEGIN TRANSACTION;

        INSERT INTO fundraising.Gift
        (
            TenantId,
            ConstituentId,
            CampaignId,
            GiftDate,
            TotalAmount
        )
        VALUES
        (
            @TenantId,
            @ConstituentId,
            @CampaignId,
            @GiftDate,
            @TotalAmount
        );

        SET @NewGiftId =
            CONVERT
            (
                BIGINT,
                SCOPE_IDENTITY()
```

```
            );

        INSERT INTO fundraising.GiftAllocation
        (
            TenantId,
            GiftId,
            FundId,
            AllocationAmount
        )
        VALUES
        (
            @TenantId,
            @NewGiftId,
            @FundId,
            @TotalAmount
        );

        COMMIT TRANSACTION;

        RETURN 0;

    END TRY
    BEGIN CATCH

        IF XACT_STATE() <> 0
        BEGIN
            ROLLBACK TRANSACTION;
        END;

        THROW;

    END CATCH;
END;
GO
```

Execute:

SQL

```
DECLARE
    @GiftId BIGINT,
    @ReturnCode INT;

EXEC @ReturnCode =
    fundraising.usp_CreateGift

        @TenantId = 1,

        @ConstituentId = 1001,

        @CampaignId = 200,

        @FundId = 300,

        @GiftDate = '2026-09-21',

        @TotalAmount = 1000,

        @NewGiftId = @GiftId OUTPUT;

SELECT
    @ReturnCode AS ReturnCode,
    @GiftId AS NewGiftId;
```

Understand the Complete Flow

EXAMPLE / FLOW

```
Application
   |
   | sends parameters
   v
Validate parameters
   |
   v
BEGIN TRY
   |
   v
BEGIN TRANSACTION
   |
   +---- Insert Gift
   |
   +---- Capture GiftId
   |
   +---- Insert GiftAllocation
   |
   v
COMMIT
   |
   v
Return NewGiftId
   |
   v
RETURN 0
```

Failure path:

EXAMPLE / FLOW

```
Error
  |
  v
CATCH
  |
  v
ROLLBACK
  |
  v
THROW
```

Stored Procedure Performance Best Practices

Stored procedures are not automatically fast.

Performance still depends on:

EXAMPLE / FLOW

```
Indexes

Statistics

Join logic

Parameter values

Row counts
```

```
Predicates

Cardinality estimates

Execution plans

Temp object choices

Transaction length

Dynamic SQL design
```

Use SARGable Filters

Less ideal:

```sql
WHERE YEAR(GiftDate) = @Year;
```

Better when possible:

```sql
WHERE GiftDate >= @StartDate
  AND GiftDate < @EndDate;
```

This gives an appropriate index on `GiftDate` a better chance to be used efficiently.

Avoid SELECT *

Less ideal:

```sql
SELECT *
FROM fundraising.Gift;
```

Prefer:

```sql
SELECT
    GiftId,
    ConstituentId,
    CampaignId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift;
```

Benefits:

EXAMPLE / FLOW

```
Clear procedure contract

Less data transfer

Safer schema changes

Better readability
```

Keep Transactions Short

Do not:

EXAMPLE / FLOW

```
BEGIN TRANSACTION

Wait for external API

Perform user interaction

Run long unrelated processing

COMMIT
```

Long transactions can hold:

EXAMPLE / FLOW

```
locks

log space

resources
```

for longer than necessary.

Do Not Hide Errors

Avoid:

SQL

```sql
BEGIN CATCH
    PRINT 'Something failed';
END CATCH;
```

and then pretending the operation succeeded.

Usually:

EXAMPLE / FLOW

```
Rollback
+
THROW
```

is safer.

Avoid Excessive Business Logic in SQL

Stored procedures are excellent for:

EXAMPLE / FLOW

```
data-centric operations

transactions

set-based transformations

database validations
```

But not every application rule belongs in SQL Server.

Your .NET service layer may be a better location for:

EXAMPLE / FLOW

```
complex workflow orchestration

external API calls

UI-specific business behavior

cross-service coordination
```

Keep responsibilities clear.

Stored Procedure vs View

Stored Procedure	View
Executed with **EXEC**	Queried in **FROM**
Accepts parameters	No normal parameters
Can modify data	Mainly query abstraction
Can contain control flow	Declarative query
Can use transactions	Cannot act like a multi-statement procedure
Can return multiple result sets	One relational result definition

Stored Procedure vs Function

A stored procedure can:

EXAMPLE / FLOW

```
Modify database state

Use transactions

Return multiple result sets

Use output parameters

Return status codes
```

A function is designed to return a value or table and has more restrictions.

Functions are a separate topic.

Stored Procedure vs CTE

Stored procedure:

EXAMPLE / FLOW

```
Persistent executable database program
```

CTE:

EXAMPLE / FLOW

```
Temporary named query expression
for one SQL statement
```

Stored Procedure vs Dynamic SQL

Stored procedure:

EXAMPLE / FLOW

```
Known stored module
```

Dynamic SQL:

EXAMPLE / FLOW

```
SQL text generated at runtime
```

A stored procedure can itself contain dynamic SQL.

Common Stored Procedure Mistakes

Mistake 1 — Using sp_ for Custom Procedures

Avoid:

EXAMPLE / FLOW

```
sp_CreateGift
```

Prefer:

EXAMPLE / FLOW

```
usp_CreateGift
```

or another team naming standard.

Mistake 2 — Not Schema-Qualifying Objects

Less ideal:

SQL

```sql
SELECT *
FROM Gift;
```

Prefer:

SQL

```sql
SELECT
    ...
FROM fundraising.Gift;
```

Mistake 3 — Wrong Parameter Types

Do not use:

EXAMPLE / FLOW

```
VARCHAR
```

for a `BIGINT` key.

Match parameter types to database columns.

Mistake 4 — No Validation

This:

EXAMPLE / FLOW

```
@PageSize = -100
```

should not be silently accepted.

Validate meaningful parameter rules.

Mistake 5 — Using RETURN for Data Values

Wrong idea:

EXAMPLE / FLOW

```
RETURN LifetimeGiving
```

Use:

EXAMPLE / FLOW

```
OUTPUT parameter
or
SELECT result
```

for business data.

Mistake 6 — Forgetting OUTPUT at Call Site

Procedure:

SQL

```
@GiftId BIGINT OUTPUT
```

Caller must use:

SQL

```
@GiftId = @Variable OUTPUT
```

to capture it.

Mistake 7 — Catching Errors but Not Rethrowing

If the operation failed, the caller needs to know.

Use:

SQL

```
THROW;
```

when appropriate.

Mistake 8 — Opening a Transaction Without Reliable Rollback

Always design:

EXAMPLE / FLOW

```
success path
and
failure path
```

together.

Mistake 9 — Dynamic SQL with Concatenated User Input

This creates SQL injection risk.

Use:

EXAMPLE / FLOW

```
sp_executesql
+
parameters
+
whitelisting for identifiers
```

Mistake 10 — Using WHILE for Set-Based Work

Avoid row-by-row processing where one SQL statement can operate on the complete set.

Mistake 11 — Assuming Procedures Are Always Faster

A poorly written procedure is still poorly written SQL.

Mistake 12 — Adding WITH RECOMPILE Everywhere

Compilation has a cost.

Use recompile only when parameter-sensitive behavior justifies it.

Mistake 13 — Huge Procedures Doing Everything

A 3,000-line procedure containing:

EXAMPLE / FLOW

```
CRUD

reporting

workflow

security

dynamic SQL

data cleanup
```

is difficult to maintain.

Design procedures around clear responsibilities.

Mistake 14 — Returning Too Many Unrelated Result Sets

Keep procedure contracts understandable for application callers.

Mistake 15 — Hidden #Temp Table Coupling

Avoid designing Procedure B so that it silently requires a temp table to have been created by Procedure A unless that pattern is very deliberate and documented.

Stored Procedure Design Best Practices

Use:

EXAMPLE / FLOW

```
Clear schema-qualified names

Meaningful parameter names

Data types matching table columns

SET NOCOUNT ON

Explicit column lists

Parameter validation
```

```
TRY...CATCH for transactional operations

SET XACT_ABORT ON where appropriate

Short transactions

THROW rather than silently hiding errors

Parameterized dynamic SQL

Least privilege

Stable result-set contracts

Comments for non-obvious business logic

Source control for procedure definitions
```

Practical Procedure Design Pattern

A clean application procedure often follows this structure:

SQL

```sql
CREATE OR ALTER PROCEDURE ...
    -- Parameters
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    -- Validation

    BEGIN TRY

        -- Optional transaction

        -- Main database work

        -- Return result/output

    END TRY
    BEGIN CATCH

        -- Rollback when needed

        THROW;

    END CATCH;
END;
GO
```

25 Interview Questions and Practical Problems

1. What is a stored procedure?

A stored procedure is a named SQL Server database program containing one or more T-SQL statements.

It can accept parameters, perform database operations, return result sets or output parameters, call other procedures, and return an integer status code.

2. Why use stored procedures?

Common reasons are:

EXAMPLE / FLOW

```
Reusable SQL logic

Transactions

Error handling

Security

Parameters

Centralized database operations

Stable application contracts
```

3. How do you create and execute a stored procedure?

Create:

SQL

```sql
CREATE PROCEDURE fundraising.usp_GetAllGifts
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        GiftId,
        TotalAmount
    FROM fundraising.Gift;
END;
```

Execute:

SQL

```sql
EXEC fundraising.usp_GetAllGifts;
```

4. What is the difference between CREATE, ALTER, and DROP PROCEDURE?

EXAMPLE / FLOW

```
CREATE
→ Create a new procedure.

ALTER
→ Modify an existing procedure.

DROP
→ Remove the procedure.
```

`CREATE OR ALTER` conveniently handles both create and update deployment cases.

5. Why should a user procedure not use the sp_ prefix?

`sp_` is associated with SQL Server system procedures.

Using it for application procedures can create naming conflicts and resolution problems.

Use a team convention such as:

EXAMPLE / FLOW

```
usp_
```

or no prefix.

6. What is an input parameter?

An input parameter passes a value from the caller into the procedure.

Example:

SQL

```
@ConstituentId BIGINT
```

used to return Gifts for one Constituent.

7. What is a default parameter?

A default parameter has a value that SQL Server uses if the caller does not supply one.

Example:

SQL

```
@PageSize INT = 25
```

8. What is an OUTPUT parameter?

An OUTPUT parameter returns a value from the procedure to the caller.

Example:

SQL

```
@LifetimeGiving DECIMAL(18,2) OUTPUT
```

Both the procedure definition and the caller capturing the value use the **OUTPUT** keyword.

9. What is the difference between RETURN and OUTPUT?

RETURN returns one integer status code.

OUTPUT parameters return business/data values and can return multiple values.

Example:

EXAMPLE / FLOW

```
RETURN 0
→ Success

@NewGiftId OUTPUT
→ Newly created GiftId
```

10. How can a procedure return data?

Three main approaches are:

EXAMPLE / FLOW

```
SELECT result sets

OUTPUT parameters

RETURN status code
```

A procedure may also return multiple `SELECT` result sets.

11. Can a stored procedure return multiple result sets?

Yes.

Multiple `SELECT` statements can produce multiple result sets.

For example:

EXAMPLE / FLOW

```
Result 1
→ Constituent

Result 2
→ Gifts

Result 3
→ Interactions
```

12. What is the difference between IF and CASE?

`IF` controls whether SQL statements execute.

`CASE` returns a conditional value inside an expression.

13. When should WHILE be used?

Use `WHILE` when sequential repetition is genuinely required.

For normal relational updates, prefer set-based SQL because it is usually simpler and more efficient.

14. Can a procedure use temporary tables?

Yes.

Example:

SQL

```
CREATE TABLE #GiftSummary (...);
```

A local temp table created inside the procedure is automatically dropped when the procedure finishes, although nested procedures called from it can access that temp table.

15. What is TRY...CATCH?

`TRY...CATCH` handles T-SQL run-time errors.

Statements execute inside `TRY`; if a catchable error occurs, SQL Server transfers control to `CATCH`.

Inside `CATCH`, functions such as `ERROR_MESSAGE()` and `ERROR_NUMBER()` describe the error.

16. What does THROW do?

`THROW` raises an error.

Inside a `CATCH` block:

SQL

```
THROW;
```

rethrows the original error.

A custom error can use:

```sql
THROW 50001,
      'Invalid value.',
      1;
```

17. How should transactions be used inside procedures?

Use a transaction when several database changes form one atomic business operation.

Example:

```
Insert Gift
+
Insert GiftAllocation
```

should either both succeed or both roll back.

Use:

```
BEGIN TRANSACTION

COMMIT

ROLLBACK

TRY...CATCH

XACT_STATE()
```

appropriately.

18. What is XACT_STATE()?

It tells you transaction state:

```
1
→ Active and committable

0
→ No transaction

-1
→ Active but uncommittable
```

This is especially useful inside `CATCH`.

19. What is dynamic SQL?

Dynamic SQL is SQL text built and executed at runtime.

Use it when SQL structure truly needs to vary.

Do not concatenate untrusted parameter values into executable SQL.

Use parameterized `sys.sp_executesql`.

20. What is the difference between EXEC() and sp_executesql?

`EXEC(@sql)` executes a dynamic string.

`sp_executesql` also supports strongly defined parameters.

For user-supplied values, parameterized `sp_executesql` is generally the preferred pattern.

21. Query Problem — Create a paged Gift procedure.

SQL

```
CREATE OR ALTER PROCEDURE fundraising.usp_GetGiftsPaged
    @PageNumber INT = 1,
    @PageSize INT = 25
AS
BEGIN
    SET NOCOUNT ON;

    IF @PageNumber < 1
        THROW 50001,
              'Invalid PageNumber.',
              1;

    IF @PageSize NOT BETWEEN 1 AND 200
        THROW 50002,
              'Invalid PageSize.',
              1;

    DECLARE @Offset INT =
        (@PageNumber - 1)
        * @PageSize;

    SELECT
        GiftId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    ORDER BY
        GiftDate DESC,
        GiftId DESC

    OFFSET @Offset ROWS
    FETCH NEXT @PageSize ROWS ONLY;
END;
```

The unique `GiftId` tie-breaker helps produce deterministic row ordering.

22. What is parameter sniffing?

At compilation time SQL Server can use procedure parameter values when estimating cardinality and selecting an execution plan.

Usually this is beneficial.

It becomes a problem when strongly different parameter values require substantially different plans.

23. What are WITH RECOMPILE and OPTION(RECOMPILE)?

Procedure level:

SQL

```
WITH RECOMPILE
```

causes the procedure to compile for executions rather than rely on a reused cached plan.

Statement level:

```sql
OPTION (RECOMPILE)
```

recompiles only the individual statement.

Use these only when there is a measured reason because recompilation consumes CPU.

24. How do procedure permissions work?

Grant:

```sql
GRANT EXECUTE
ON OBJECT::fundraising.usp_GetConstituentGifts
TO FundraisingAppRole;
```

This allows the principal to execute the procedure.

Ownership chaining may allow access to referenced tables without granting direct access to those tables, but dynamic SQL requires special attention because ordinary ownership chaining does not automatically cover it.

25. Full Interview Problem — Create Gift Safely

Requirement:

```
Accept Gift details.

Validate the amount.

Create Gift.

Create GiftAllocation.

Return new GiftId.

Roll back everything on failure.
```

Core structure:

```sql
CREATE OR ALTER PROCEDURE fundraising.usp_CreateGift
    @TenantId BIGINT,
    @ConstituentId BIGINT,
    @FundId BIGINT,
    @GiftDate DATE,
    @TotalAmount DECIMAL(18,2),
    @GiftId BIGINT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    IF @TotalAmount <= 0
    BEGIN
        THROW 50001,
              'TotalAmount must be positive.',
              1;
    END;

    BEGIN TRY

        BEGIN TRANSACTION;
```

```
        INSERT INTO fundraising.Gift
        (
            TenantId,
            ConstituentId,
            GiftDate,
            TotalAmount
        )
        VALUES
        (
            @TenantId,
            @ConstituentId,
            @GiftDate,
            @TotalAmount
        );

        SET @GiftId =
            CONVERT
            (
                BIGINT,
                SCOPE_IDENTITY()
            );

        INSERT INTO fundraising.GiftAllocation
        (
            TenantId,
            GiftId,
            FundId,
            AllocationAmount
        )
        VALUES
        (
            @TenantId,
            @GiftId,
            @FundId,
            @TotalAmount
        );

            COMMIT TRANSACTION;

            RETURN 0;

        END TRY
        BEGIN CATCH

            IF XACT_STATE() <> 0
            BEGIN
                ROLLBACK TRANSACTION;
            END;

            THROW;

        END CATCH;
    END;
```

This single example combines:

EXAMPLE / FLOW

```
Input parameters

OUTPUT parameter

Parameter validation

INSERT
```

```
Transaction

TRY...CATCH

XACT_STATE

ROLLBACK

THROW

RETURN status code
```

Final Revision Map

EXAMPLE / FLOW

```
STORED PROCEDURE
      |
      +---- Parameters
      |       |
      |       +---- Input
      |       +---- Default
      |       +---- OUTPUT
      |
      +---- Returned data
      |       |
      |       +---- Result sets
      |       +---- OUTPUT parameters
      |       +---- RETURN code
      |
      +---- Logic
      |       |
      |       +---- Variables
      |       +---- IF / ELSE
      |       +---- CASE
      |       +---- WHILE
      |
      +---- Temporary processing
      |       |
      |       +---- #Temp table
      |       +---- Table variable
      |
      +---- Error handling
      |       |
      |       +---- TRY
      |       +---- CATCH
      |       +---- ERROR_* functions
      |       +---- THROW
      |
      +---- Transactions
      |       |
      |       +---- BEGIN TRAN
      |       +---- COMMIT
      |       +---- ROLLBACK
      |       +---- SAVE TRANSACTION
      |       +---- XACT_STATE
      |
      +---- Dynamic SQL
```

```
|       |
|       +---- EXEC()
|       +---- sp_executesql
|       +---- Dynamic filters
|       +---- Dynamic ORDER BY
|
+---- Performance
|       |
|       +---- Execution plans
|       +---- Parameter sniffing
|       +---- PSP / OPPO
|       +---- RECOMPILE
|
+---- Security
        |
        +---- GRANT EXECUTE
        +---- Ownership chaining
        +---- EXECUTE AS
```

Final Easy Memory Rules

EXAMPLE / FLOW

Stored Procedure
→ Reusable executable database program

EXAMPLE / FLOW

Input parameter
→ Data goes INTO procedure

EXAMPLE / FLOW

OUTPUT parameter
→ Data comes OUT of procedure

EXAMPLE / FLOW

RETURN
→ Integer status code

EXAMPLE / FLOW

SELECT
→ Result set

EXAMPLE / FLOW

TRY...CATCH
→ Handle errors

EXAMPLE / FLOW

THROW
→ Raise/rethrow error

EXAMPLE / FLOW

Transaction
→ All-or-nothing database operation

EXAMPLE / FLOW

#Temp table
→ Temporary intermediate data

EXAMPLE / FLOW

```
sp_executesql
→ Parameterized dynamic SQL
```

EXAMPLE / FLOW

```
OFFSET / FETCH
→ Page-based pagination
```

EXAMPLE / FLOW

```
WITH RECOMPILE
→ Recompile procedure plan
```

EXAMPLE / FLOW

```
OPTION (RECOMPILE)
→ Recompile one statement
```

EXAMPLE / FLOW

```
GRANT EXECUTE
→ Allow caller to execute procedure
```

The most important overall rule is:



Use a stored procedure when the database needs a reusable, parameterized operation—not simply because you want to wrap every SQL query. Design each procedure around one clear responsibility, keep its transaction and error behavior predictable, parameterize dynamic SQL, and tune it from actual execution evidence rather than assuming that stored procedures are automatically faster.